

Dining cryptographers

Dining Cryptographers

Three cryptographers gather around a table for dinner. The waiter informs them that the meal has been paid for by someone, who could be

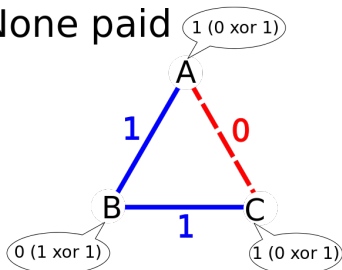
- one of the cryptographers or
- the National Security Agency (NSA)

The cryptographers respect each other's right to make an anonymous payment, but want to find out whether the NSA paid.

Solution

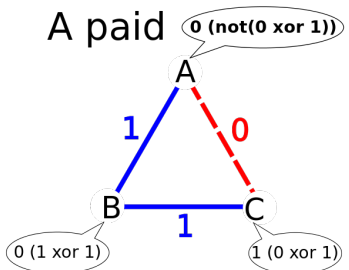
- 1 Pairwise toss a coin behind the menu to establish a shared one-bit secret.
- 2 Announce a bit:
 - If didn't pay: XOR of the shared bits
 - If paid: opposite of the XOR
- 3 Computes the XOR of the three bits announced.
 - If the result is 1: one of them paid
 - Otherwise: none of the paid.

None paid



$$1 \text{ xor } 1 \text{ xor } 0 = 0$$

A paid



$$1 \text{ xor } 0 \text{ xor } 0 = 1$$

Secret messaging

The DC problem is basically the following:

- Given 3 participants.
- One of them might send an information.
- Pairwise shared random.

Goal: send message without while keep the sender anonymous.

Generalization

How to generalize the protocol for

T1 n participant

T2 longer messages.

T3 multiple participants want to send message.

Solution:

T1 The same protocol works

T2 Repeat the 1-bit algorithm, or do everything over a finite field.

T3 Problem: if multiple participants sends message, they ruin each others message.

Goal: Protocol that satisfies T1, T2, T3.

Generalization

How to generalize the protocol for

T1 n participant

T2 longer messages.

T3 multiple participants want to send message.

Solution:

T1 The same protocol works

T2 Repeat the 1-bit algorithm, or do everything over a finite field.

T3 Problem: if multiple participants sends message, they ruin each others message.

Goal: Protocol that satisfies T1, T2, T3.

Multiple sender

- Problem: Multiple senders cannot send the message at the same time.
- Multiple rounds: Each sender sends its message in a different round.
- Question: How to handle message sending?

Send in a random round

Many rounds, each participant choose one randomly and send the message in that round.

Slot reservation

- Time slots: Rounds to send messages.
- Reservation phase: before message sending, participants can reserve a time slot for sending their message.
- Types: reservation vector, reservation map, superposed receiving

Multiple sender

- Problem: Multiple senders cannot send the message at the same time.
- Multiple rounds: Each sender sends its message in a different round.
- Question: How to handle message sending?

Send in a random round

Many rounds, each participant choose one randomly and send the message in that round.

Slot reservation

- Time slots: Rounds to send messages.
- Reservation phase: before message sending, participants can reserve a time slot for sending their message.
- Types: reservation vector, reservation map, superposed receiving

Superposed sending

Similar to secure aggregation, put with possibly vector inputs. The difference is that all parties behave like a server.

Superposed sending

- Participants: $P_1, \dots, P_n$, Input: $x_i \in \mathbb{F}^\ell$ for participant P_i .
- Goal: Compute $s = \sum_i x_i$

Algorithm

- Setup: share key between P_i and P_j : k_{ij}^ℓ
- P_i computes: $s_i = x_i + \sum_{i < j} k_{ij} - \sum_{i > j} k_{ij}$, and sends s_i to every other participants.
- Output: P_i computes $s = \sum_i s_i$

Reservation vector technique

Reservation phase

- Let ℓ be number of time slots. P_i choose a random index $r_i \in \{1, \dots, \ell\}$.
 - v_i is the vector of length ℓ with 1 in the r_i th position and 0 in all others.
 - The participants jointly perform a superposed sending over $\mathbb{F}_2$ where the input of P_i is v_i .
 - The output is the reservation vector v .
-
- If $v_{r_i} = 1$, then the slot reservation of P_i was successful.
 - If $v_{r_i} = 0$, then the reservation of the slot of P_i was unsuccessful. P_i misses the next message sending phase, and P_i has to try again in the next reservation phase.

Reservation vector technique

Message sending optimization

- If $v_j = 0$ for an index j , then the j th round can be skipped in the message-sending phase.
- If $v_{r_i} = 1$, and there are t 1 in v in the first $r_i - 1$ coordinates, then P_i sends its message in the $t + 1$ th round.

Analysis

- 1 reservation for slot: $\checkmark$
- 0, 2, 4 reservation for a slot: $\checkmark$
- 3, 5, 7 reservation: the message sending fails but this occurs with only a small probability

Reservation map

The reservation vector technique, but the computation is done modulo m (and not modulo 2).

- Advantage: Can detect if 3 participants reserved the same slot.
- Disadvantage: Increases the vectors bit-length.

Superposed receiving

Message sending algorithm based on superposed sending. In each round a subset of participants send their message in such a way that after n rounds it is possible to compute every message using the previous values.

- 0 G additive group, P_i 's message is $m_i \in G$ -t.
- 1 The participants jointly perform the superposed receiving algorithm with input $(1, m_i)$ to P_i . The output is $(n, m = \sum_i m_i)$
- 2 Every participant computes the average: m/n
- 3 Next round only those sends message (which is not $(0,0)$), whose value is under the average $m_i < m/n$

Superposed receiving

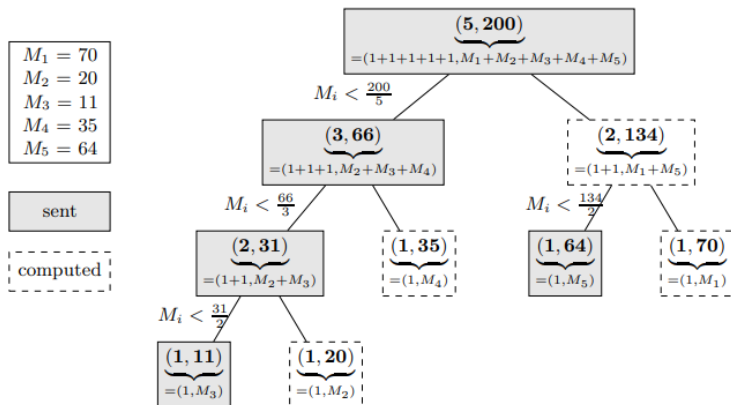


Figure 3.5: Superposed Receiving

Slot reservation with superposed receiving

The superposed receiving technique can be used to slot reservation (for example if the messages are long).

- 1 Each participant chooses a random message, $P_i \leftarrow r_i$.
- 2 Participants perform the superposed receiving algorithm with input r_i for P_i .
- 3 If r_i is the t th message, then P_i reserved the t th slot.

Message sending

Graph of DC

- Not every pair of participants has a shared key.
- The shared keys can be represented in a graph.
- Vertices: participants, Edges: have shared key
- Sometimes participants reveal their share keys. If two participants disagree, remove the edge from the graph.

DC protocol

- Reservation phase: participants have a chance to reserve a time slot.
- Message sending phase: in every round participants perform a superposed sending.
 - If a participant reserved that slot: his input is his message
 - For other participants: their input is 0.

Disruption and investigation

Attack - Disruption

- disruptor send a message on a time slot that he didn't reserved.
- hard to detect

If a participant detects that the message in his time slot is different from the one he sent, he can start an investigation.

Investigation

- Each participant reveals their shared keys, their input and the messages they received in that round.
- They show the others how they computed their message.
- The shared key of every two participants must be the same.
- The input must be 0, if the participant didn't send message in that round.

Disruption and investigation

Attack - Disruption

- disruptor send a message on a time slot that he didn't reserved.
- hard to detect

If a participant detects that the message in his time slot is different from the one he sent, he can start an investigation.

Investigation

- Each participant reveals their shared keys, their input and the messages they received in that round.
- They show the others how they computed their message.
- The shared key of every two participants must be the same.
- The input must be 0, if the participant didn't send message in that round.

Investigation

- ① If a participant sent different message to different participant: detectable
 - ② If two participants used different shared key: detectable, but impossible to tell, who cheated
 - ③ If a participant cheated on computing his message: detectable unless he lies about one his shared key or his input.
 - ④ If multiple participants input is not zero (so they want to send message in that round): impossible to tell, who this slot belonged to.
- In cases 1 or 3, the participant must be excluded from the protocol.
 - In case 2, we remove the shared key of these two participants.
 - In case 4: we need more tools.

Trap

- Instead of sending a message, make a trap in the time slot.
 - Commitment before message sending (encrypted): I send y as trap in slot x .
 - If in slot x not y is the message, open commitment and start investigation.
-
- If a disruptor sends a fake message in a trap slot, then the owner of the slot can prove his ownership, so the disruptor cannot lie about it.
 - Disruptor can lie about its message but he had to change one of his keys. This key must be removed.

Commitments to shared keys:

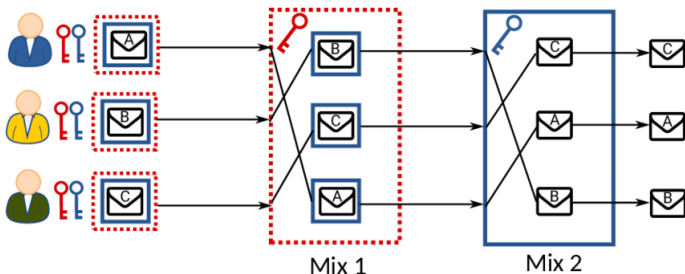
- Key commitment: h hash function (or one-way function).
Each participant computes a joint hash of all of its keys for every round sends it to every other parties.
- Participants cannot lie about their keys anymore.

Trap reservation reveal:

- If there is an investigation because of a trap, the participants first investigate the reservation phase.
- Disruptors cannot forge fake traps.

Mix network

Protocol for anonymous message sending:



- Users send their message encrypted to the mixer.
- Mixer forwards the message to the recipient.
- More mixer: more security.
- Security:
 - Recipient: Can read the message but don't know the sender.
 - Mixer: Might know the sender, but have no information about the content.

Message sending

Message sending with three participant, Alice (sender), Bob (recipient), M (mixer)

1 Alice sends a message to Bob. $K_B(message, R_0)$

2 Message through M : $K_M(R_1, K_B(R_0, message), B)$

3a If Alice wants a response: $K_M(S_1, A), K_x$ (K_x is a one-time-key)

3b Bob's response: $K_M(S_1, A), K_x(S_0, response)$, M forwards $K_x(S_0, response)$ to Alice.

Whole message from Alice to Bob:

$$K_M(R_1, K_B(R_0, message, K_M(S_1, A), K_x), B) \rightarrow \\ \rightarrow K_B(R_0, message, K_M(S_1, A), K_x)$$

Response from Bob to Alice:

$$K_M(S_1, A), K_x(S_0, response) \rightarrow A, K_x(S_0, response)$$

Message sending

Message sending with three participant, Alice (sender), Bob (recipient), M (mixer)

1 Alice sends a message to Bob. $K_B(message, R_0)$

2 Message through M: $K_M(R_1, K_B(R_0, message), B)$

3a If Alice wants a response: $K_M(S_1, A), K_x$ (K_x is a one-time-key)

3b Bob's response: $K_M(S_1, A), K_x(S_0, response)$, M forwards $K_x(S_0, response)$ to Alice.

Whole message from Alice to Bob:

$$K_M(R_1, K_B(R_0, message, K_M(S_1, A), K_x), B) \rightarrow \\ \rightarrow K_B(R_0, message, K_M(S_1, A), K_x)$$

Response from Bob to Alice:

$$K_M(S_1, A), K_x(S_0, response) \rightarrow A, K_x(S_0, response)$$

Message sending

Message sending with three participant, Alice (sender), Bob (recipient), M (mixer)

- 1 Alice sends a message to Bob. $K_B(message, R_0)$
- 2 Message through M : $K_M(R_1, K_B(R_0, message), B)$
- 3a If Alice wants a response: $K_M(S_1, A), K_x$ (K_x is a one-time-key)
- 3b Bob's response: $K_M(S_1, A), K_x(S_0, response)$, M forwards $K_x(S_0, response)$ to Alice.

Whole message from Alice to Bob:

$$K_M(R_1, K_B(R_0, message, K_M(S_1, A), K_x), B) \rightarrow \\ \rightarrow K_B(R_0, message, K_M(S_1, A), K_x)$$

Response from Bob to Alice:

$$K_M(S_1, A), K_x(S_0, response) \rightarrow A, K_x(S_0, response)$$

Mix network attacks

Attack:

- Mixer is not honest - multiple independent mixers

Attacks by traffic monitoring:

- Timing attack: - batching
- Length of message - fixed length message

Mix network attacks

Attack:

- Mixer is not honest - multiple independent mixers

Attacks by traffic monitoring:

- Timing attack: - batching
- Length of message - fixed length message